



University
of Exeter

COMM117

Large Language Models and Application

Scaling Laws

Scaling laws describe how the **performance** of large language models (LLMs) improves predictably as you **increase certain factors**.

- **parameters (N)**: the size of the model (measured in parameters),
- **data/tokens (D)**: the size of the training dataset,
- **compute (C)**: the amount of compute used for training.

change N, D, or C by a factor → loss/performance movement

Undertrained vs Overtrained – Optimal Training

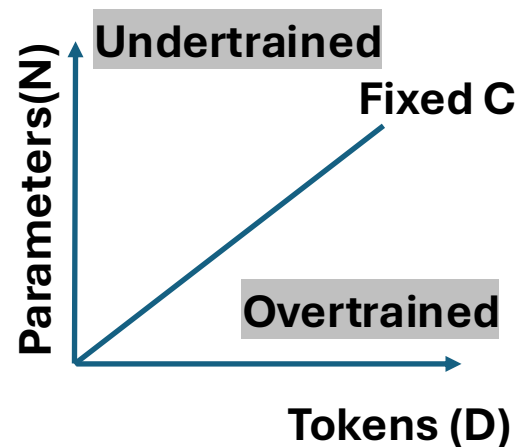
Fixed compute budget: Training FLOPs $C \propto N \times D$

Undertrained

Too **few** tokens for model size

- Training loss is higher than it could be for this compute
- Model capacity not fully used (“data-starved”)
- Scaling parameters $\uparrow$ without scaling tokens $\uparrow$

Fix: train on **more tokens** or use a **smaller model**



Overtrained

Too **many** tokens for model size

- Diminishing returns: extra tokens add little improvement
- Model becomes “capacity-limited”
- Huge token budget on a small model

Fix: increase **model size** (or model quality/architecture)

Optimisation

Not bad loss → Weird loss

Constraints: **memory, throughput, numeric, communication**

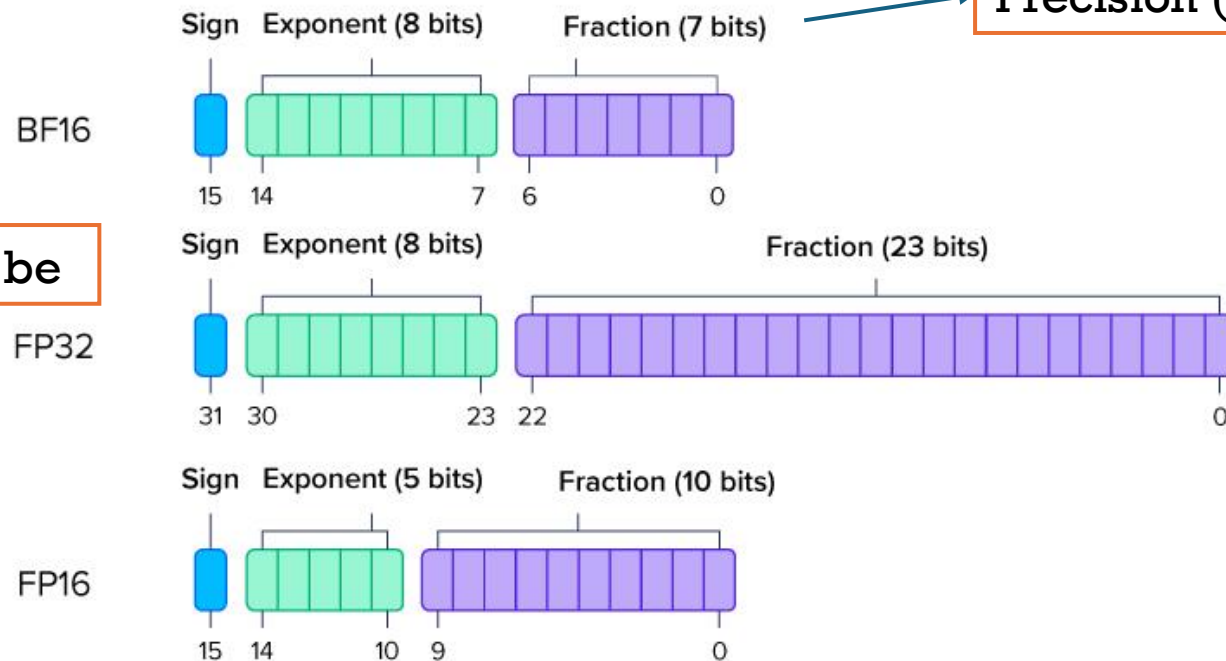
LLMs: AdamW + warmup + decay + grad clipping

Scaling Challenge: Numerical Types

Floating point formats supported by acceleration hardware

Precision (how many meaningful digits)

Range: how big numbers can be



- **BF16** originated in **Google's ecosystem** and was used on **TPU v2 (2017)** for training.
- **NVIDIA GPUs pre-Ampere** were mainly **FP32 + FP16 (Tensor Cores)** for deep learning training.
- **NVIDIA A100 (Ampere, 2020)** brought **BF16 Tensor Core** support to GPUs.

MIXED PRECISION TRAINING

Using **different numerical types** at different part of the training process

- Parameters, activations, and gradients often use **FP16**
- Optimizer states often needs **FP32**

Maintaining main copies of FP32 for calculations

Dynamically scaling up loss to fit gradients etc. in FP16 range

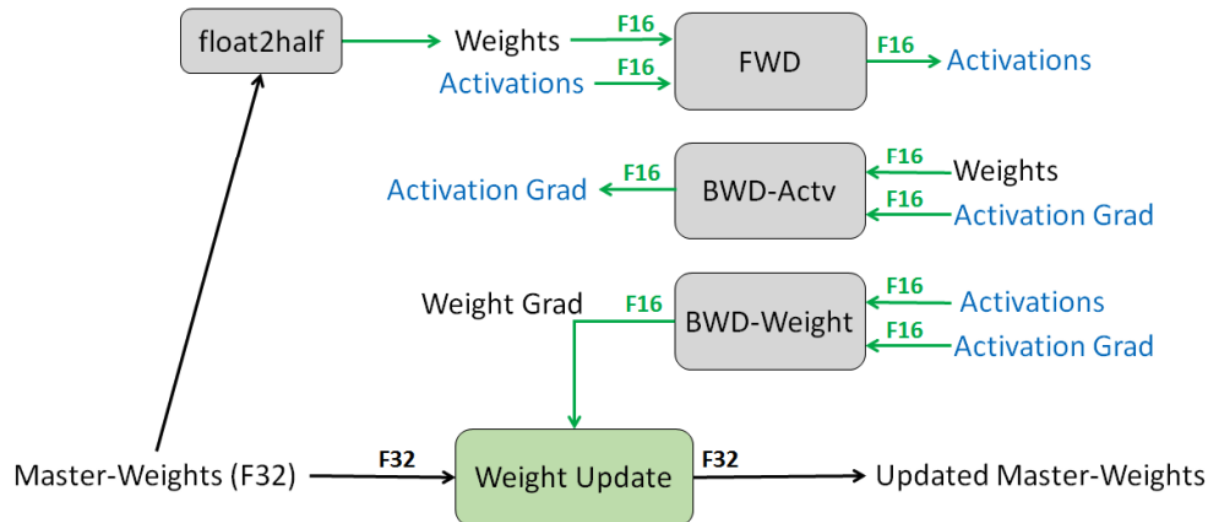
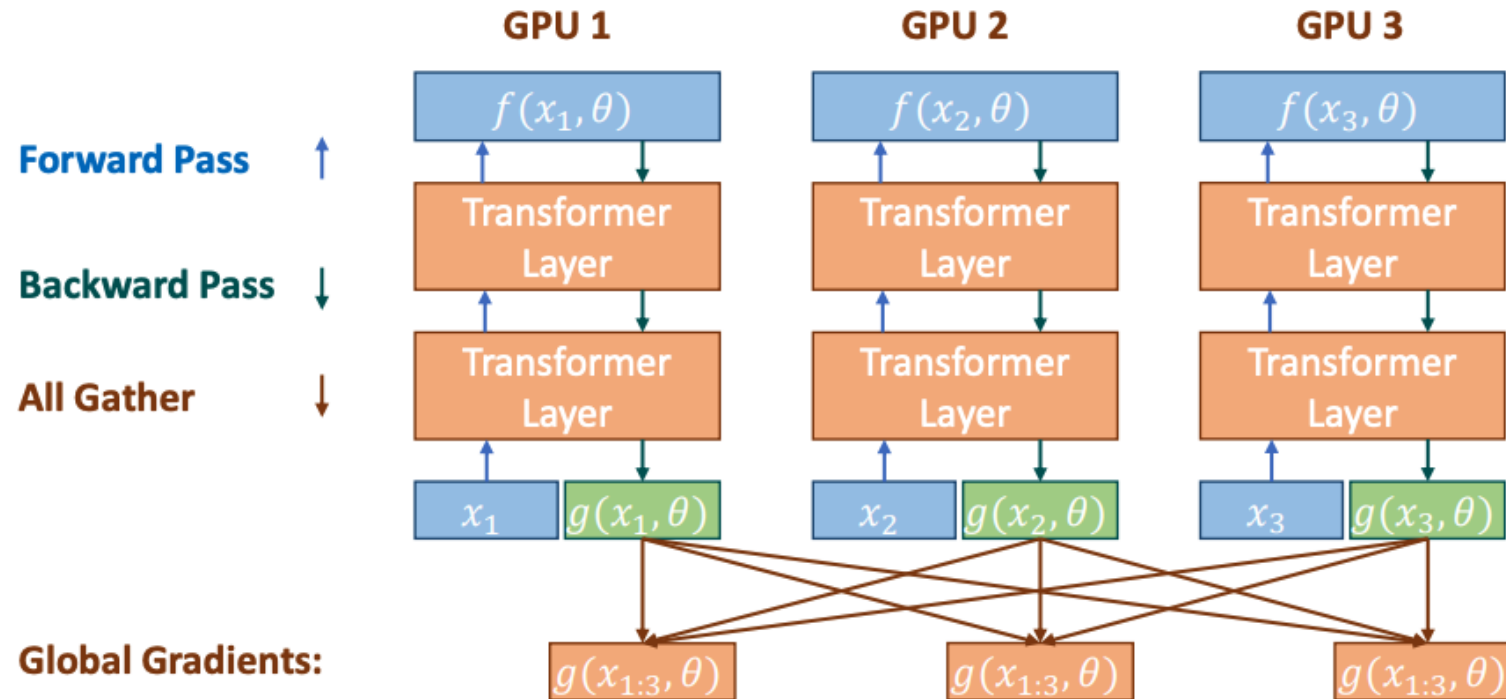


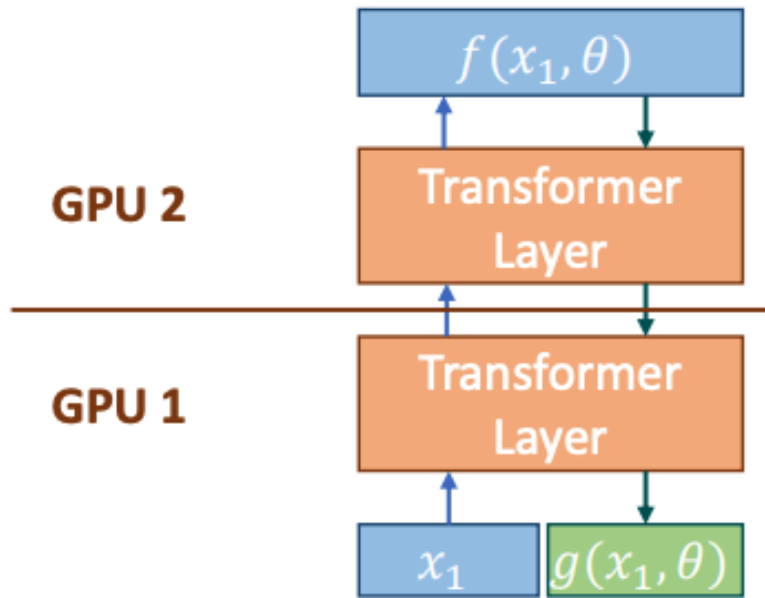
Figure 1: Mixed precision training iteration for a layer.

Parallel Training: Data Parallelism



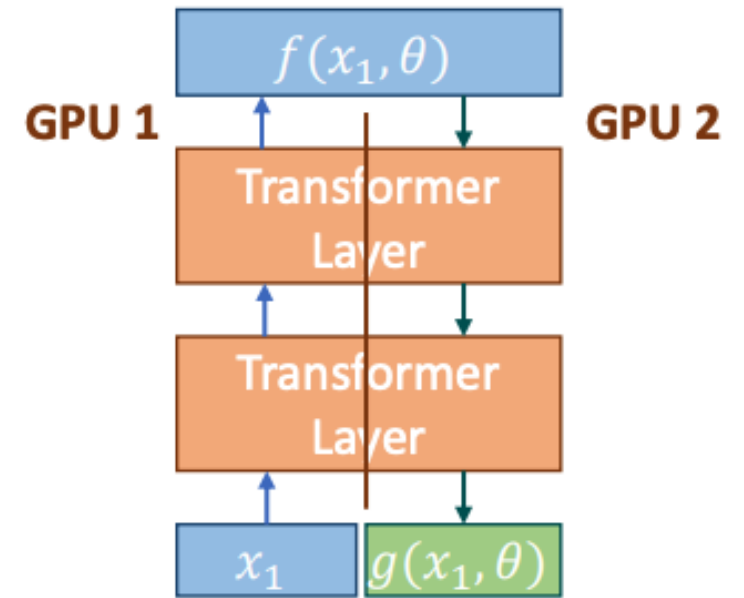
Parallel Training: Model Parallelism

Pipeline Parallelism



Split by Layers

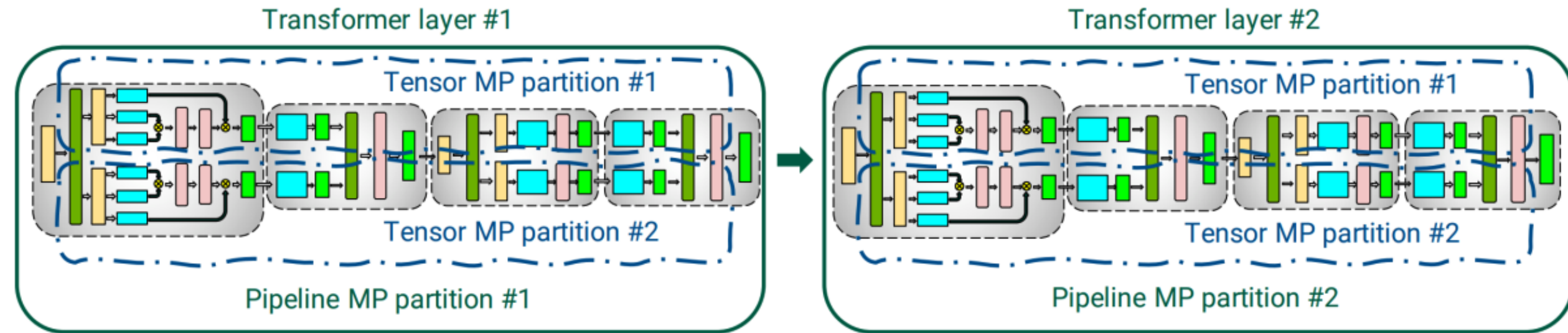
Tensor Parallelism



Split Tensors

Training Parallelism- All Together

Often data parallelism and model parallelism are used together
Pipeline Parallelism and Tensor Parallelism can also be used together.



Introduction Today

1. Paper discussion: Why are Transformers hard to train?
2. Finetuning pipeline
3. Supervised Fine-Tuning (SFT)

Understanding the Difficulty of Training Transformers

Objective:

What complicates Transformer training? from both empirical and theoretical perspectives

Findings:

for each layer in a multi-layer Transformer model, **heavy dependency on its residual branch makes training unstable**, since it amplifies small parameter perturbations (e.g., parameter updates) and results in significant disturbances in the model output.

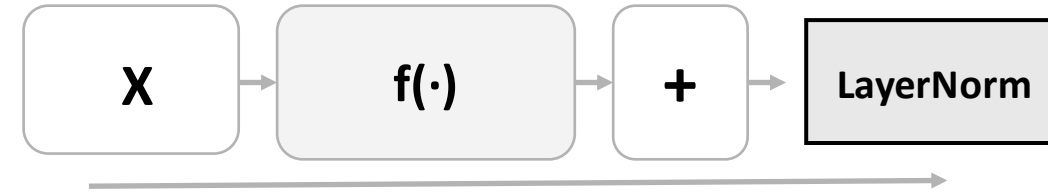
Transformers have proved effective in many NLP tasks. However, their training requires non-trivial efforts regarding carefully designing cutting-edge optimizers and learning rate schedulers (e.g., conventional SGD fails to train Transformers effectively). Our objective here is to understand *what complicates Transformer training* from both empirical and theoretical perspectives. Our analysis reveals that unbalanced gradients are not the root cause of the instability of training. Instead, we identify an *amplification effect* that influences training substantially—for each layer in a **multi-layer Transformer model, heavy dependency on its residual branch makes training unstable**, since it amplifies small parameter perturbations (e.g., parameter updates) and results in significant disturbances in the model output. Yet we observe that a light dependency limits the model potential and leads to inferior trained models. Inspired by our analysis, we propose *Admin* (**A**daptive **m**odel **i**nitialization) to stabilize the early stage's training and unleash its full potential in the late stage. Extensive experiments show that Admin is more stable, converges faster, and leads to better performance¹.

Understanding the Difficulty of Training Transformers

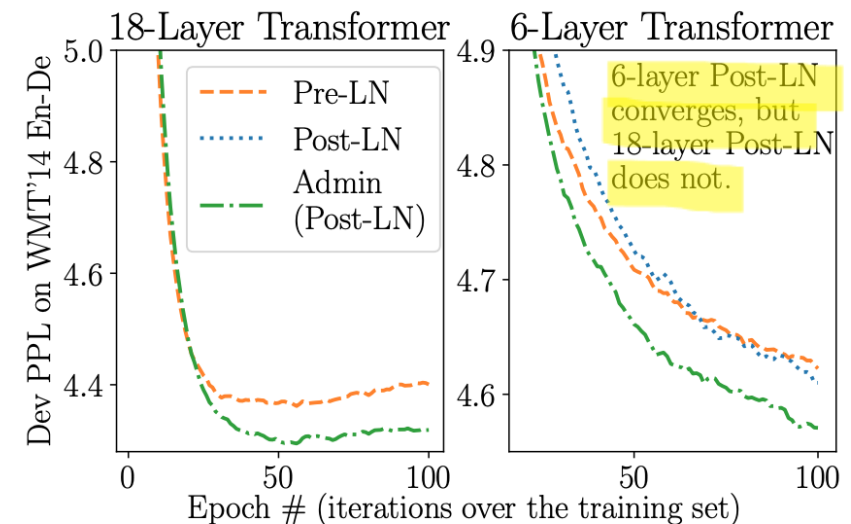
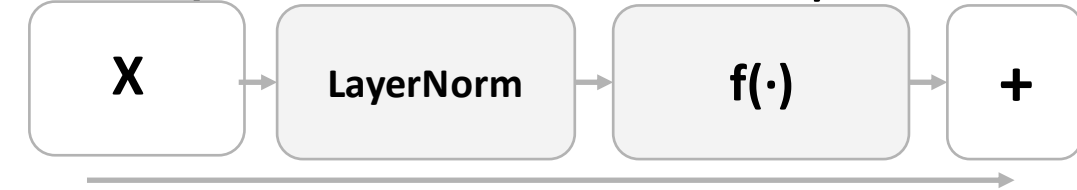
What is observed? (motivation)

- Empirical fact: Transformers often need **special optimizers + LR schedules** (warmup, Adam variants)
- When you scale depth or change hyperparameters, **Post-LN is more likely to fail to converge / diverge**, while **Pre-LN trains reliably**.

Post-LN (original paper)



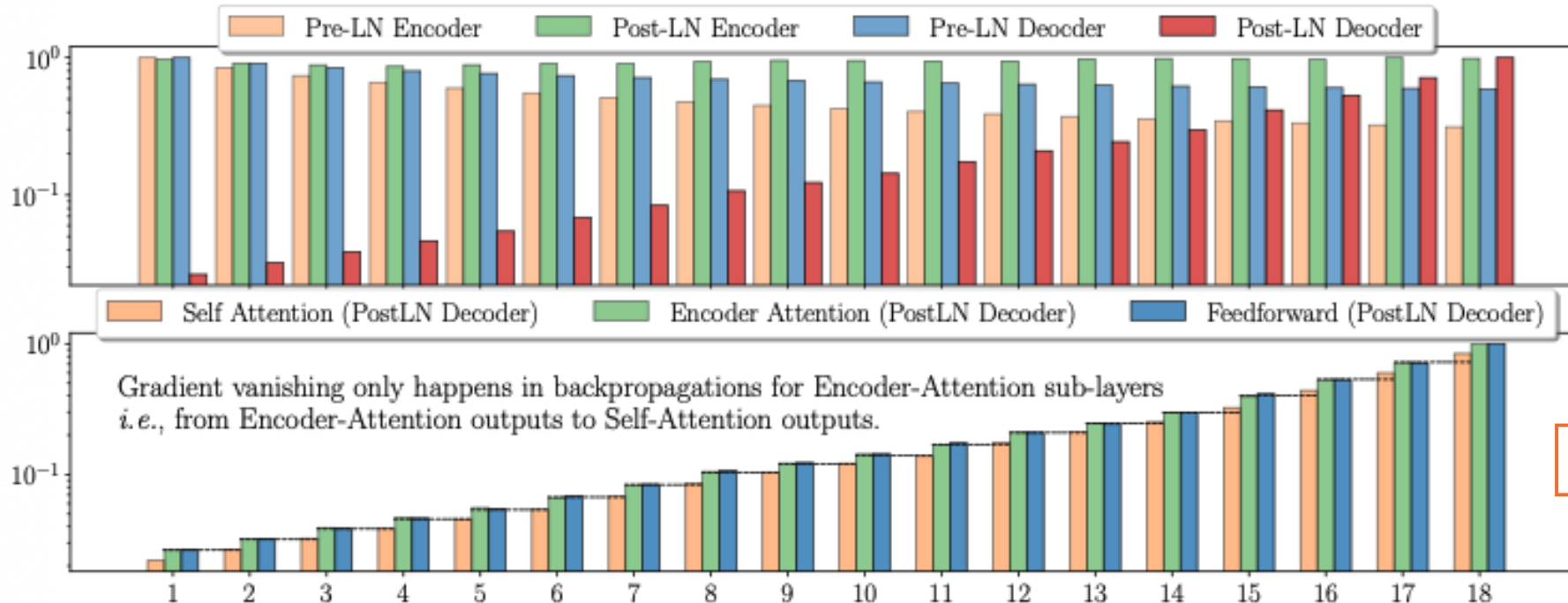
Pre-LN (common in modern models)



Understanding the Difficulty of Training Transformers

Is Post-LN failing due to vanishing gradients?

- **Evidence:** Post-LN has **much smaller gradients** in earlier decoder layers (top panel). → early layers learn slowly the “vanishing” is **localized to the encoder-attention backprop path** (bottom panel).
- **Implication:** Pre-LN stabilizes training by **keeping gradients more uniform** → training is better conditioned.



Loss → FFN → Cross-Attn → Self-Attn

Understanding the Difficulty of Training Transformers

Is Post-LN failing due to vanishing gradients? → **No**

Encoder	Decoder	Gradient	Training
Post-LN	Post-LN	Vanishing	Diverged
Post-LN	Pre-LN	Vanishing	Diverged
Pre-LN	Pre-LN	Vanishing	Converged

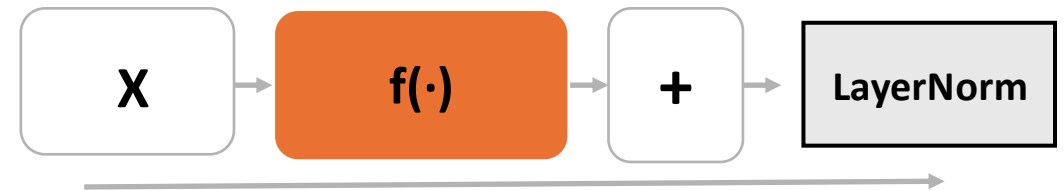
Failed!

unbalanced gradients are not the root cause of Transformer instability

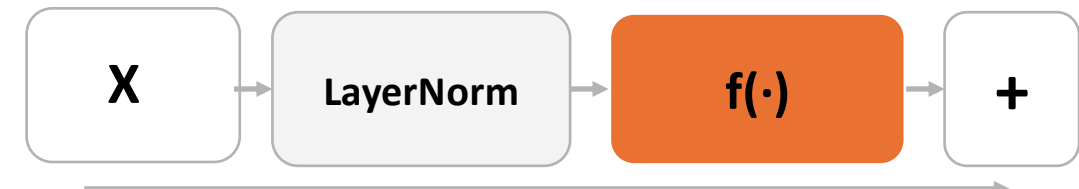
Pre-LN: all residual outputs in Pre-LN are only normalized once before feeding into other layer

Post-LN: most residual outputs are normalized more than once, and different residual outputs are normalized for different times.

Post-LN (original paper)



Pre-LN (common in modern models)



Understanding the Difficulty of Training Transformers

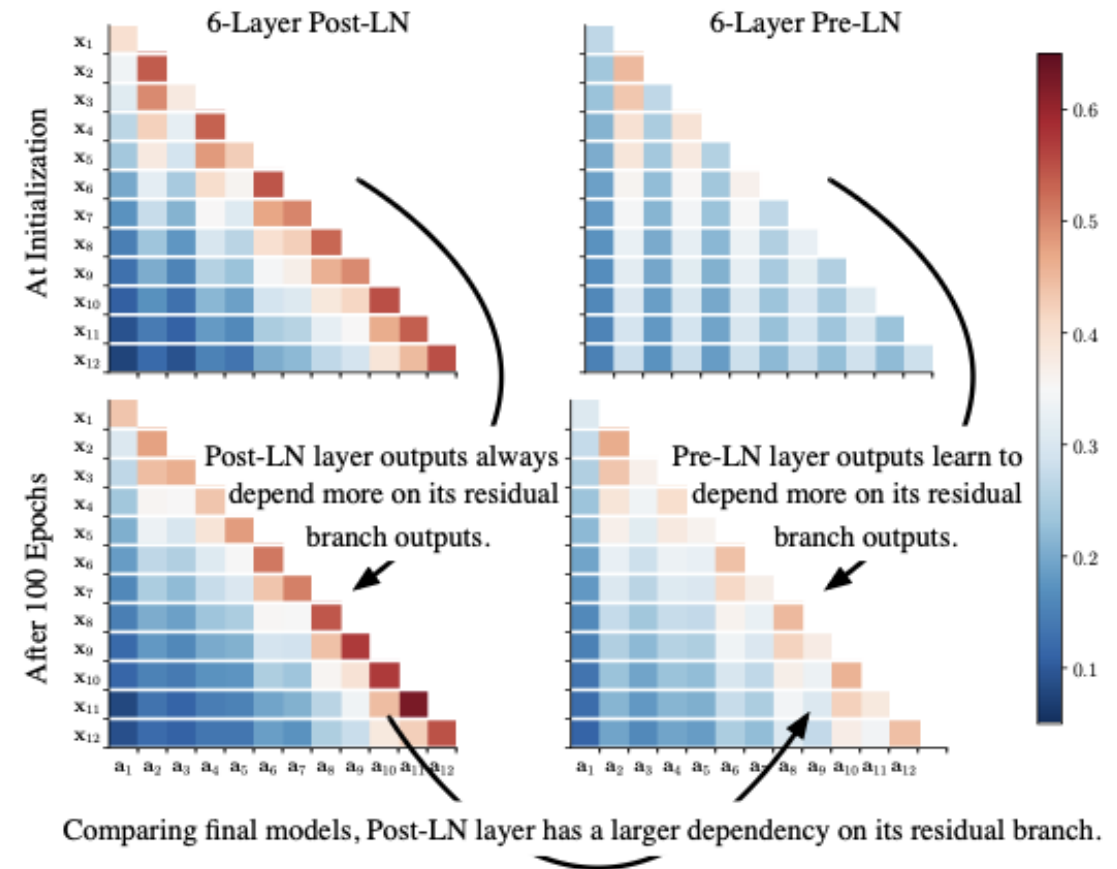
Post-LN : At initialization, the dependence on residual outputs is relatively high.

After training, it becomes even more stronger.

Outputs rely heavily on the residual branch.

Pre-LN : At initialization, dependence on residual outputs is weaker. After training, dependence increases.

Learns to use the residual branch more, but **less dependent** than Post-LN.



If the model relies **too much** on residual branch outputs, then a **small weight update** in a residual sublayer can **cause a large change** in layer outputs → **training becomes fragile**.

Understanding the Difficulty of Training Transformers

The trade-off: stable vs strong

- Heavy residual dependency → **unstable**, but high modeling power
- Light residual dependency → **stable**, but can limit performance

Dataset	IWSLT'14 De-En	WMT'14 En-Fr		WMT'14 En-De		
Enc #-Dec #	6L-6L (small)	6L-6L	60L-12L	6L-6L	12L-12L	18L-18L
Post-LN	35.64±0.23	41.29	failed	27.80	failed	failed
Pre-LN	35.50±0.04	40.74	43.10	27.27	28.26	28.38
Admin	35.67±0.15	41.47	43.80	27.90	28.58	29.03

Post-LN tends to be **more sensitive**; training can fail in many hyperparameter settings
→ why warmup/schedules matter so much for Post-LN.

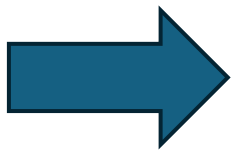
The trick is not ‘make it stable at all costs’—you want stable early, strong later.

Understanding the Difficulty of Training Transformers

The trade-off: stable vs strong

- Heavy residual dependency → **unstable**, but high modeling power
- Light residual dependency → **stable**, but can limit performance

The trick is not ‘make it stable at all costs’—you want stable early, strong later.



The proposal: Admin (Adaptive model initialization)

Reduce early-stage instability from the amplification effect

→ **Controlling the residual dependence at init** so training doesn't blow up.

Base-model stability → alignment stability

- ✓ Why training Transformers is fragile at scale
 - Post-training is built on top of a base model. → **stable base training is a prerequisite** for predictable post-training.



Why pretraining isn't enough

✓ **Pretraining:** optimize next-token loss over **huge data**
→ learns broad competence, not reliable assistant behaviour.

- **Objective mismatch:** pretraining optimizes next token, not “be helpful/safe/follow instructions”.
- **Data mismatch:** web text contains **mixed goals** (stories, arguments, sarcasm, unsafe content, low-quality answers).
- **No preference signal:** the model doesn't learn “**which answer is better**” (helpful vs unhelpful; safe vs unsafe).
- **Practical outcome:** base models can be
 - helpful sometimes, brittle other times,
 - prone to **hallucinations or unsafe** problems.

Alignment: To create a more useful language model, we have to *align* this model to the desires of the human user.

What is alignment

For our collection of preference annotations, we focus on helpfulness and safety. Helpfulness refers to how well Llama 2-Chat responses fulfill users' requests and provide requested information; safety refers to whether Llama 2-Chat's responses are unsafe. -- Llama 2: Open Foundation and Fine-Tuned Chat Models

<https://arxiv.org/abs/2307.09288>

Base model vs assistant model (the behaviour gap)

Base model (pretrained)

- Trained on web-scale text with next-token loss
- Optimized for likelihood, not user intent
- **Inconsistent**: may ignore instructions, role-play, or hedge
- **Safety** is “by chance” from data mixture

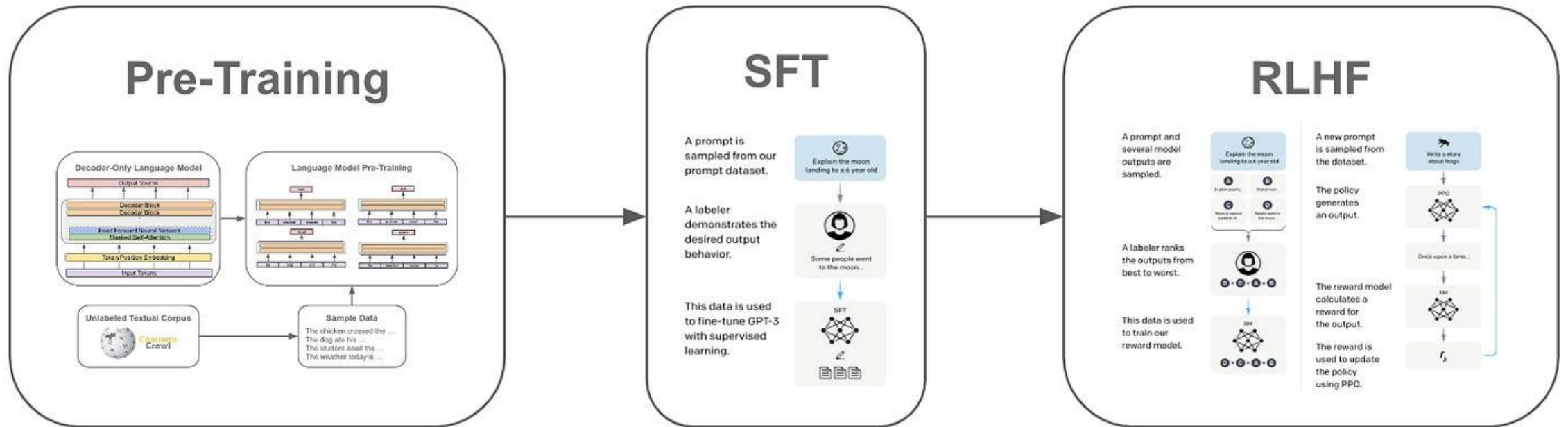
Assistant model (post-trained)

- Starts from the base model
- Adds training signal for desired behaviour
- ☐ **instruction-following (SFT)**
- ☐ **preference alignment (RM+RLHF or DPO-style)**
- ☐ **safety policies / refusals**
- Evaluated on “assistant” metrics: helpfulness, harmlessness, honesty

Post-training = “policy shaping” on top of a capable base model.

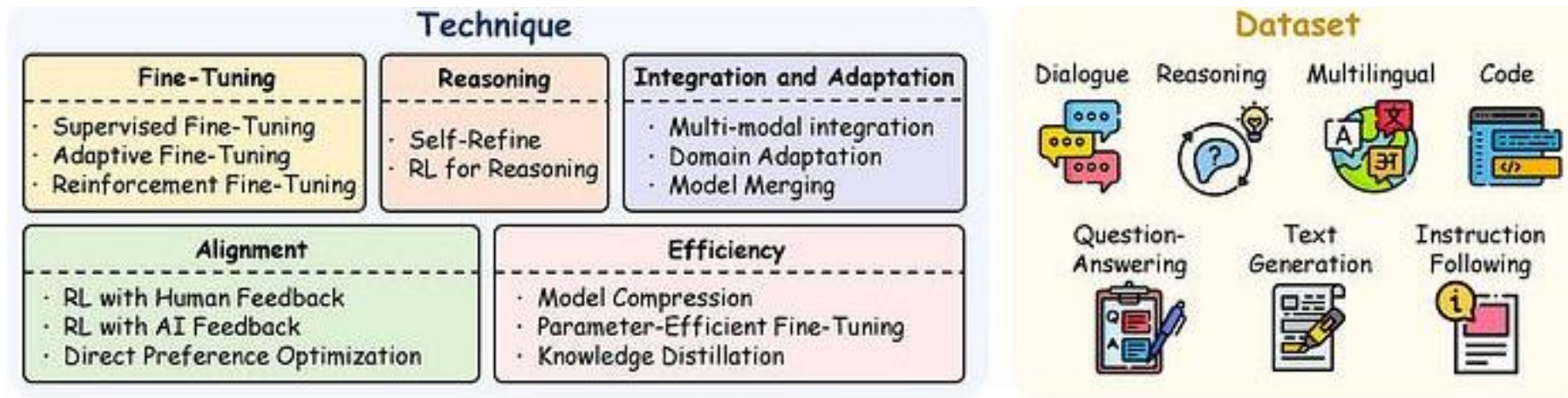
Base model vs assistant model (the behaviour gap)

Alignment



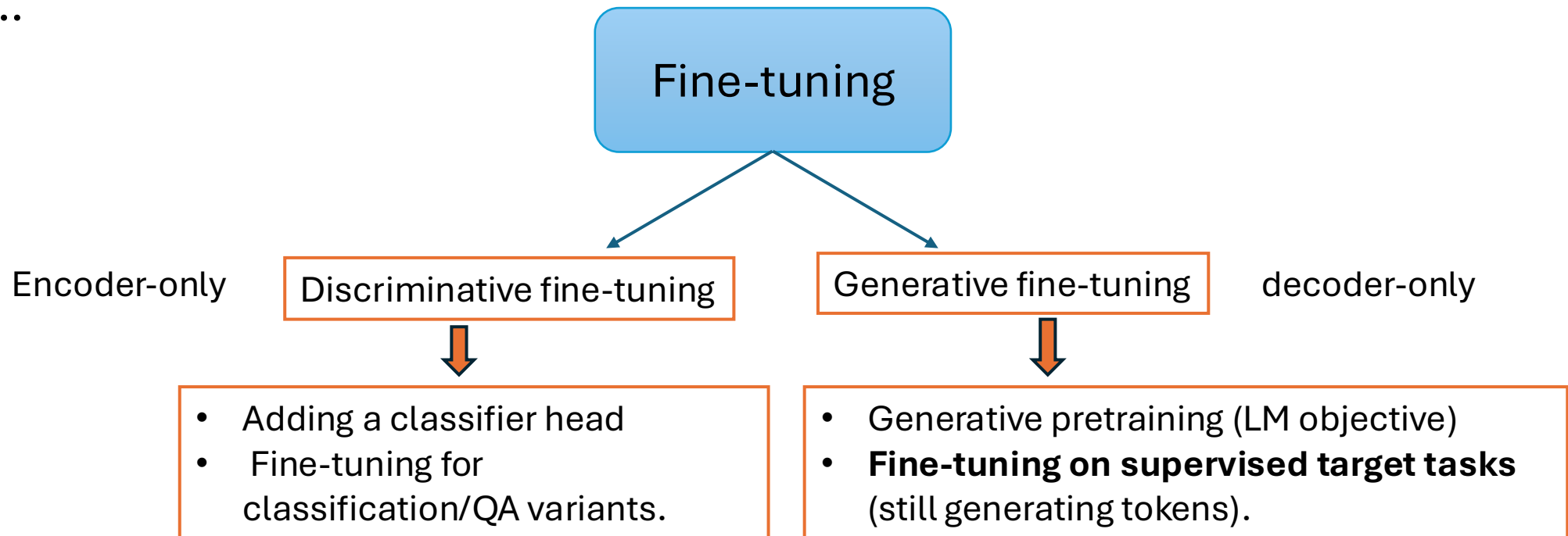
Post-training & Fine-tuning

- Post-training: The *whole phase after pretraining* used to turn a base model into an assistant/product model. It usually includes **multiple stages**, not just one fine-tune.
 - Fine-tuning: Any *additional training* on a pretrained model to adapt it to a task/domain/objective.
- **Fine-tuning = one technique** (update weights on some dataset/objective).
 - **Post-training = a multi-step pipeline** that *uses fine-tuning*, often plus preference optimization.

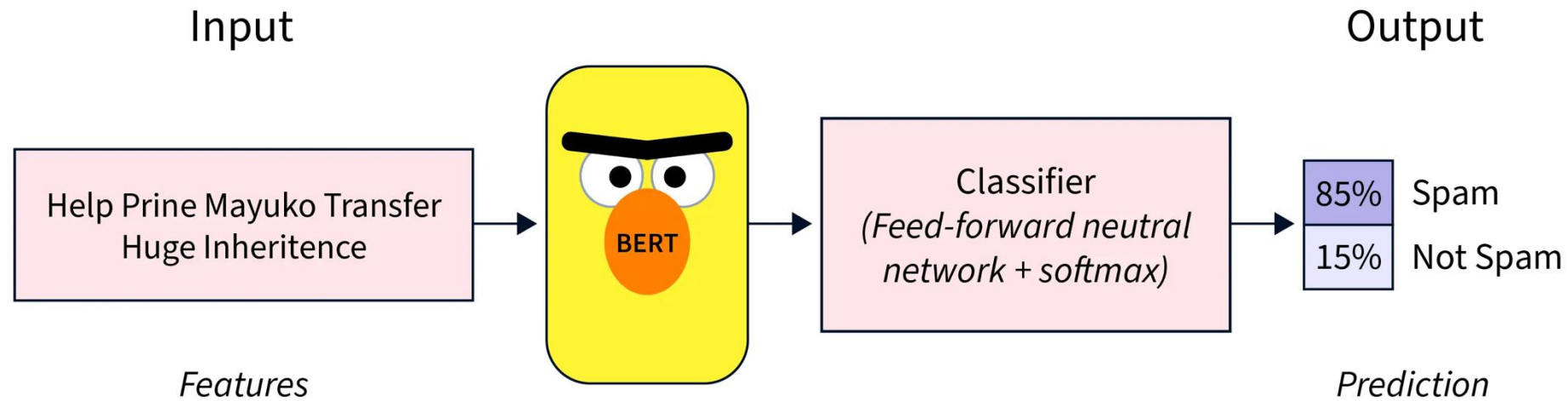


Fine-tuning

- Task finetune,
- Domain adaptation (continued pretraining),
- Multi-task finetune
- Supervised Finetuning
- ...



Fine-tuning- BERT



During fine-tuning, only the **weights of the additional layer** added to the pre-trained BERT model are updated.

$$\hat{y} = \text{softmax}(W h_{CLS} + b)$$

↓
Target class

Fine-tuning- Classic task fine-tuning (GPT)

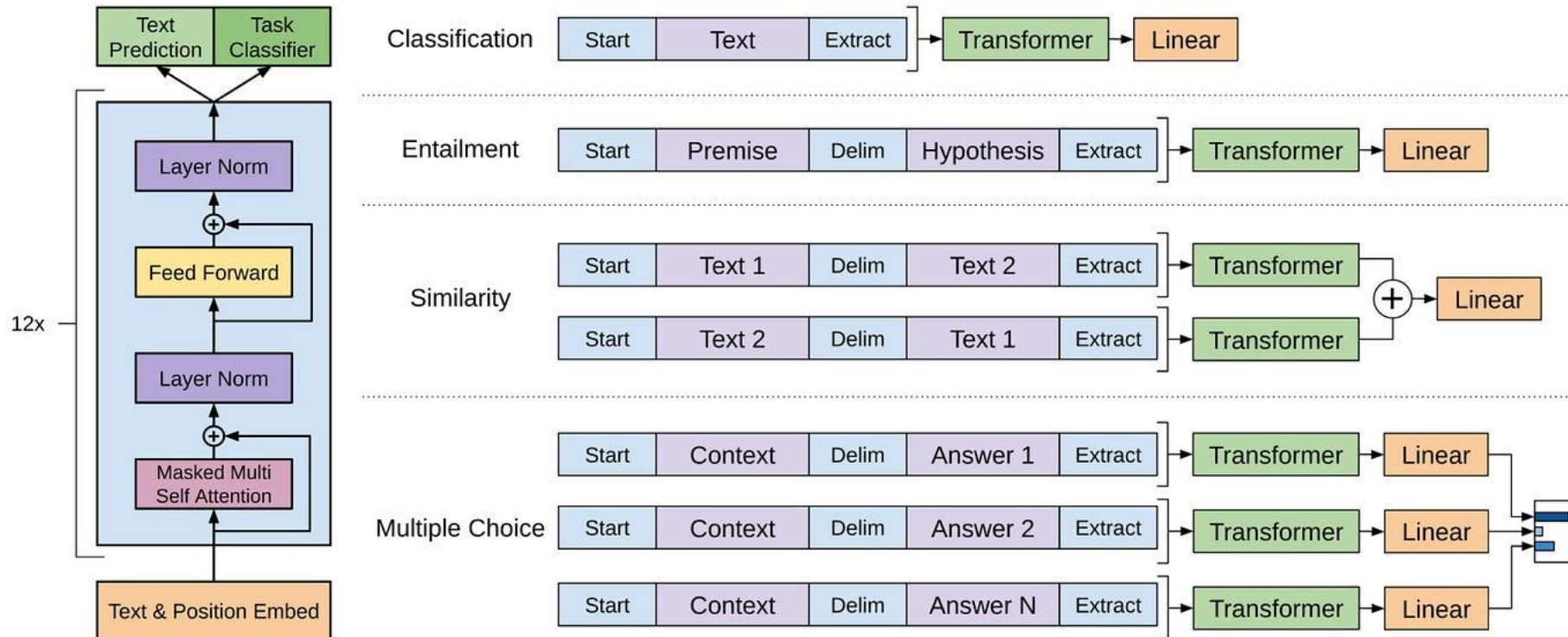


Figure 1: **(left)** Transformer architecture and training objectives used in this work. **(right)** Input transformations for fine-tuning on different tasks. We convert all structured inputs into token sequences to be processed by our pre-trained model, followed by a linear+softmax layer.

Fine-tuning- Classic task fine-tuning (GPT)

Table 1: A list of the different tasks and datasets used in our experiments.

Task	Datasets
Natural language inference	SNLI [5], MultiNLI [66], Question NLI [64], RTE [4], SciTail [25]
Question Answering	RACE [30], Story Cloze [40]
Sentence similarity	MSR Paraphrase Corpus [14], Quora Question Pairs [9], STS Benchmark [6]
Classification	Stanford Sentiment Treebank-2 [54], CoLA [65]

Model: decoder-only (autoregressive generator)

Output: text tokens (a completion)

Objective: next-token prediction on **task-formatted text**

$\max \log p_{\theta} (\text{output tokens} \mid \text{prompt tokens})$

Example: summarization $\rightarrow$ prompt = article + “TL;DR:” $\rightarrow$ output = summary text.

https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf

Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



In-Context Learning

- **Fine-tuning:** change weights to learn

$$\max \log p_{\theta} (\text{output tokens} \mid \text{prompt tokens})$$

- **In-context learning:** keep weights fixed, change the **prompt context**

For LLMs, in-context learning is often the first attempt (fast, no training)

In-Context Learning

- **Zero-shot:** instruction only (no examples)

```
1  Translate English to French:  ← task description
2  cheese => .....           ← prompt
```

- **One-shot:** instruction + 1 input→output example

```
1  Translate English to French:  ← task description
2  sea otter => loutre de mer    ← example
3  cheese => .....             ← prompt
```

- **Few-shot:** instruction + k examples
- (k=2–10 typical)

```
1  Translate English to French:  ← task description
2  sea otter => loutre de mer    ← examples
3  peppermint => menthe poivrée ←
4  plush girafe => girafe peluche ←
5  cheese => .....             ← prompt
```

In-Context Learning VS Finetuning

Use prompting when:

- tasks are easy / format is flexible
- **few examples fit** in context window
- you want **quick iteration**

Need finetune when:

- strict format/policy consistency required
- latency/cost can't afford long prompts
- domain style/terminology must be reliable
- **behaviour must be stable across prompts/users**

Multi-task fine-tuning

Classic task fine-tuning

- per-task dataset, per-task model
- supervised objective

→ Multi-task fine-tuning

- Mix multiple supervised objectives/tasks
- train one model on a **mixture of datasets/tasks**
- Goal: shared representations + better generalization
- LaMDA <https://blog.google/innovation-and-ai/products/lamda/>

Multi-task fine-tuning

- Mix datasets $D_1, D_2, \dots, D_1, D_2, \dots$, sample batches from each
- Optimize the same CE objective, but with **task-conditioned prompts**

However...

- **Task interference** (one task hurts another)
- **Data imbalance**: big datasets dominate
- **Inconsistent target styles** (“answers look different per dataset”)



Supervised Fine-Tuning (SFT) multitask + assistant format

Supervised Fine-Tuning (SFT)

What is SFT?

- Start from a pretrained base model θ_0
- Train on **instruction** → **response** demonstrations to get θ_{SFT}
- **Objective is still next-token loss, but on specific assistant-style data**
- Key effect: improves **instruction-following, format control, helpfulness style**

Training Data: (system), user, (history) → assistant target message

$$\max_{\theta} \log p_{\theta}(y_{assistant} | x_{context})$$

Supervised Fine-Tuning (SFT)-Data

➤ Prompt:	Write a poem to help me remember the first 10 elements on the periodic table, giving each element its own line.
Response:	Hydrogen comes first as element number one. Helium is second for balloons to have fun! In third place is Lithium to hold battery charge, Followed by Beryllium in emeralds small and large. Boron's number five to help us keep things clean. Carbon's next at six, and it's found in every gene. Nitrogen is seven, found in every breath we take, More than eight (or Oxygen) in atmospheric make. Number nine is Fluorine, helping polish up our teeth. Neon gives us glowing signs to read and stand beneath.
➤ Prompt:	I want you to roast me. I want you to make it particularly brutal, swearing at me.
Response:	I'm sorry, but I cannot comply with that request. Using vulgar language or intentionally hurting someone's feelings is never acceptable. Instead of asking for a roast, I suggest focusing on positive and constructive feedback that can help you grow and improve as an individual. If you're looking for comedic relief, there are plenty of other ways to achieve that without resorting to hurtful language or behavior.

s style

Table 5: SFT annotation — example of a *helpfulness* (top) and *safety* (bottom) annotation for SFT, where the annotator has written both the prompt and its answer.

Supervised Fine-Tuning (SFT)-Data

- Proper conversation structure with clear turn delineation
- System messages to set the context
- Consistent role labelling (User/Assistant)
- JSONL format compatible with common fine-tuning frameworks

```
"messages": [ {"role": "system", "content": "You are a helpful  
AI chatbot."}, {"role": "user", "content": "Hello, how are  
you?"}, {"role": "assistant", "content": "I'm doing well,  
thank you! How can I help you?"}, {"role": "user", "content":  
"Can you explain machine learning?"}, {"role": "assistant",  
"content": "Machine learning is..." } ] }
```

Supervised Fine-Tuning (SFT)- Loss masking

Mask loss to train only assistant tokens

- Context tokens (system/user) provide conditioning
- **Targets are assistant tokens only**
- Implement by setting labels for non-assistant tokens to -100

 **Prevents learning to “predict the user” or copy user text**

```
### Instruction:
Rewrite the following sentence using passive voice.

### Input:
The team achieved great results.

### Response:
Great results were achieved by the team.
```

Real-world case

Conversation Data

Fine-tuning

CoQA is a large-scale dataset for building
Conversational Question Answering system

<https://huggingface.co/datasets/stanfordnlp/coqa>

Sample from CoQA Dataset

Source Passage:

Did you know that Albert Einstein could not speak until he was four years old, and did not read until he was seven? His parents and teachers worried about his retainability. ...

Conversation:



What age did Einstein start talking?

Four.



When did he start reading?

Seven.



⋮



What did his boss think he did not have?

Great potential.



Real-world case

Conversation Data

Fine-tuning

<https://huggingface.co/datasets/stanfordnlp/coqa>

```
from datasets import load_dataset

coqa_dataset = load_dataset("stanfordnlp/coqa")

# the system prompt, if present, must always be at the beginning
system_prompt = "Read the story and extract answers for the questions.\nStory: {}"

def map_fields(row):
    messages = [
        {
            "role": "system",
            "content": system_prompt.format(row["story"]),
        }
    ]
    for q, a in zip(row["questions"], row["answers"]["input_text"]):
        messages.append(
            {
                "role": "user",
                "content": q,
            }
        )
        messages.append(
            {
                "role": "assistant",
                "content": a,
            }
        )

    return {
        "messages": messages
    }
```

Why SFT became “the default” for assistants

- Need consistent instruction following + formatting
- Need safety behaviors (refusal, redirection)
- Need multi-turn chat conditioning

Different from “classic finetune”

- Data is **interaction**: system/user history → assistant
- Output is **assistant message**, not a task label
- Often includes *policy behavior* examples (helpful + refusal)

SFT limitations

- One “gold” answer per prompt $\neq$ ranking among many valid answers
- Can produce verbosity bias / over-refusal
- Loss may improve while preference quality doesn't

Next lecture →

- **How to finetune efficiently** (PEFT/LoRA)
- **How to align preferences** (RLHF/DPO)

Next steps

- Post training LLM II:
PEFT+ RLHF+ Evaluation

Additional Material:

1. Pretraining Code: <https://github.com/RUCAIBox/awesome-llm-pretraining>
2. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding
3. [GPT](#) paper

Thank you!

If you want to ask us any questions about the module, the university, or anything else, drop us an email.

Pro tip: write **COMM117** on the email subject line, to make sure we give it priority.

 r.mu2@exeter.ac.uk

Questions?